Parallel Patterns in OpenCL (Weeks 5 -8)

The code templates for this tutorial are available on the module's Blackboard site as a ZIP file. Please download the file and extract it before starting the tutorial, as it will be used as the starter template for all exercises. In this tutorial, we will explore three core concepts in parallel programming: the OpenCL kernel execution model, parallel map operations as implemented in OpenCL, and stencil computations. These topics build directly on material introduced in the lectures, beginning with the Week 3 lectures.

NOTE: Compile and run the program using the **VS Code integrated terminal with Git Bash**.
The first command uses g++ to compile the source file, link it against the OpenCL library, and produce an executable. The second command runs the compiled program. Replace example.cpp and example.exe with the filename of the workshop task you are working on

- g++ example.cpp -o example.exe -I"C:\OpenCL-SDK\include" -lOpenCL
- ./example.exe

Additional linker flags:

- -lgdi32 -luser32 -lkernel32 are needed for CImg display (Windows).
- -lpng -lz are needed for CImg PNG support (if you have PNG images).

1 Kernel Execution Model (Week 5)

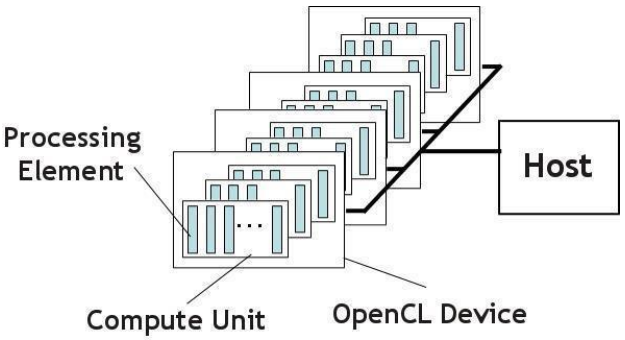

Figure 1 Each OpenCL device consists of at least 1 Compute Unit, which in turn contains one or more Processing Elements (after https://github.com/HandsOnOpenCL).

Let's now look in more detail at the inner workings of OpenCL kernel execution. As mentioned in the previous tutorial (**Week 1 - 4**), a "device" will run its code on "computing units", which in turn consists of "processing elements" (see Fig. 1). Each separate kernel execution is performed on a processing element, and in OpenCL terminology it is called a "work item". You can think about work items as separate threads. In our basic example (see **01_kernel_execution.cpp**), we have used vectors of 10 length, so the kernel launch in this case will consist of 10 work items.

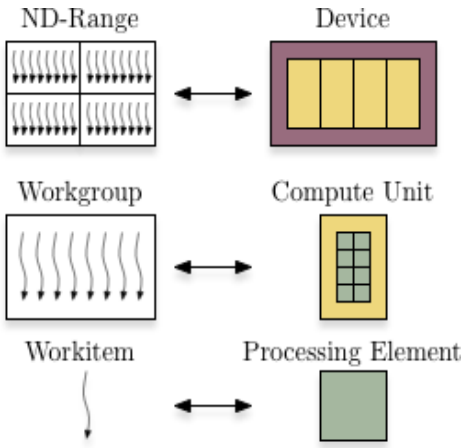

Figure 2 The hierarchy of execution: a problem is executed on a device, a workgroup on a compute unit and a work item on a processing element (after http://mygsoc.blogspot.co.uk/).

Each computing unit consists of a fixed number of processing elements, which naturally limits the number of **work items** which might be executed at the same time. In addition, the **work items** often need to communicate between each other (e.g. share the same variable, report the intermediate results, etc.) and therefore OpenCL introduces a concept of "**work groups**", which are collections of **work items** executed by a single compute unit. Finally, to enable work on larger problems, the device will launch as many **work groups** as possible at the same time, depending on the number of the existing Compute units. This process will be repeated until all the necessary work items associated with the problem are completed (see Fig. 2).

Let us see some examples to get a better intuition about these terms. For that purpose, we will need to be able to inspect the inner workings of individual kernels. A simple debugging facility is provided by the printf function, which can be used to output kernel variables used by each work item. The formatting used by this function is exactly the same as in printf used in other languages (see https://en.wikipedia.org/wiki/Printf_format_string for more details). The following code snippet shows how to use this function for printing out the idvariable from our simple vector addition kernel:

```
int id = get_global_id(0); printf("work item id =
%d\n", id);
```

The tasks in this section use the host program in `01_kernel_execution.cpp` and device/kernel codes in the file `vector_add.cl`. Ensure these files are opened in visual studio code as we will modify and add new codes to them.

1. Add the code above to the kernel codes (`vector_add.cl`). If you run the program, you will see the id values for each consecutive work item being displayed. We can also now better understand the purpose of the `get_global_id` function: it returns a unique id for each work item from 0 to N-1 where N is the total number of work items required to solve a particular problem. Be careful with the `printf` function though – if you use it on larger problems the function might interfere with the execution of kernels and even freeze your computer.

2. Run the modified kernel on different devices and note the order of the execution of work items. Is it the same for each device and each run?

In general, **work items** are executed in an arbitrary order so one cannot make any assumptions about which work item is executed first.

OpenCL specifies a number of functions similar to `get_global_id` which help to identify different launch parameters (see https://www.khronos.org/registry/OpenCL/specs/opencl-1.2.pdf#page=244 for more details). For example, `get_local_id` returns a work item id within a specific work group. The values vary from 0 to M-1 where M is the size of a work group. This size can be determined by calling `get_local_size`.

3. Now, add the following lines to your code to your kernel code (vector_add.cl) and run the program again and note the differences between local and global ids but also work group size for different devices.

```
if (id == 0) { // perform this part only once i.e. for work item 0 printf("work
       group size %d\n", get_local_size(0));
}

// Each work-item's index *inside* its work-group
int loc_id = get_local_id(0);

// Print mapping between global and local IDs
printf("global id = %d, local id = %d\n", id, loc_id); // do it for each work item
```

4. Print out the total number of work items for a given problem (just once) and a work group id for each work item. Note the differences between devices.

```
if (id == 0){
   printf("\n total work-items = %d\n\n", global_size);
}

printf("global id = %d, group id = %d\n", id, group_id);
```

The work group size in our example varies depending on which device it is running on. This is because our kernel execution command has left the fourth parameter empty (**cl::NullRange**), which is signalling to OpenCL that this value should be set automatically. It is possible however to define the work group size (or local size) manually when calling the kernel from the host:

```
queue.enqueueNDRangeKernel(kernel_add, cl::NullRange,cl::NDRange(vector_elements),
cl::NDRange(local_size));
```

In OpenCL 1.2, the input data size is required to be divisible by the work-group size, meaning that not all local sizes are permitted. This restriction is removed in OpenCL 3.0, which allows greater flexibility in handling arbitrary data sizes. We will discuss techniques for managing non-divisible workloads in later workshops; for now, examples will focus on cases where this condition is naturally satisfied.

Different kernels will have different preferred work group sizes (due to device architecture, memory usage, etc.). The recommended values for the work group size can be identified by reading a kernel property called `CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE`. This parameter can be accessed in the following way in the host code:

```
size_t preferredMultiple =
kernel.getWorkGroupInfo<CL_KERNEL_PREFERRED_WORK_GROUP_SIZE_MULTIPLE>(device);

std::cout << "Preferred WG multiple: " << preferredMultiple << std::endl;
```

5. Modify the host codes (`01_kernel_execution.cpp`) to include the above codes, which get the preferred work group size. Query recommended minimum work-group size multiple

This value is the smallest work group size suggested, but its multiples are also possible, up to the maximum work group size specified by another kernel parameter which can be accessed in the following way:

```
size_t maxWGSize = kernel.getWorkGroupInfo<CL_KERNEL_WORK_GROUP_SIZE>(device);
std::cout << "Max WG size for kernel: " << maxWGSize << std::endl;
```

6. Add the above code to the host code to query maximum work-group size allowed for this kernel.

7. We will now specify the work group size (or local size) manually when calling the kernel from the host. To do this we will choose a valid local (work-group) size by starting from the device's preferred alignment. If this value exceeds the maximum work-group size supported by the device, reduce it to the maximum allowable size.

```
size_t localSize = preferredMultiple;
if (localSize > maxWGSize)
     localSize = maxWGSize;

std::cout << "   Using local size: " << localSize << std::endl;
```

8. Run the vector addition kernel on different devices for the work group size equal to the value specified by the preferred work group size parameter and note the differences in local and group ids. You will need to manually adjust the length of the input vector, so it is divisible by the work group size.

The work groups play an important role in applications which require work items to communicate with each other.

2 Parallel Pattern – Map (Week 6)

Parallel programming patterns define standard abstract components from which larger algorithms and programs can be built. One of the most straightforward parallel patterns is the Map (see Fig. 3).

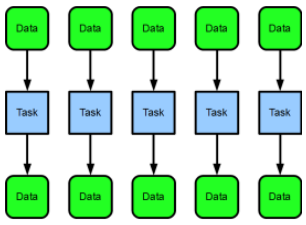

Figure 3 The Map parallel pattern (after http://parallelbook.com/sites/parallelbook.com/files/SC13_20131117_Intel_McCool_Robison_Reinders_Hebenstreit.pdf).

The map pattern describes the same computation performed on different data without the need for communication between work items. So far in these workshops, we have been working solely with the map pattern, even if we have not mentioned that explicitly. Let us now see how to apply the map pattern to solve practical problems.

In this tutorial, we will now apply the Map pattern to solve problems in physics simulation. This is not intended as a full physics simulation tutorial; instead, we will use these operations as general-purpose examples of highly parallel computation.

The template provided in the workshop folder is configured to load particle simulation data stored as binary files. Each particle is represented by three floating-point values, for example, as velocity components in the x, y, and z directions. The `common.hpp` file contains utility functions shared across all tasks in this tutorial and is included in each host file. The two datasets can be found in the workshop folder under the `dataset/` subfolder:

- `small_data.bin` (100,000 particles) is recommended for development and correctness testing
- `large_data.bin` (10,000,000 particles) is recommended for profiling and performance analysis

In the `.cpp` file, you can choose your dataset: `small_data.bin` (1,000 particles) for debugging, or `large_data.bin` (100,000 particles) for profiling..

2.1. Apply Gravity

In many physical systems, a constant external acceleration (e.g. gravity) influences velocity.

$$v = v + g \cdot \Delta t$$

Which demonstrates how independent state updates can be applied uniformly using Map.

a) You need to select the appropriate dataset, which currently has been set to `small_data.bin` in the host code (`02_gravity.cpp`).

```
std::string filename = "small_data.bin";
std::vector<float> velocities;
if (!loadSimulationData(filename, velocities)) {
        return -1;
}
```

b) In the kernel codes (`gravity.cl`), you are to complete the kernel, `apply_gravity` that updates velocities. Detail instructions in implementing it can be found as comments in the kernel codes. The `apply_gravity` kernel is to update each particle's velocity using $v_{\text{new}} = v + g \cdot dt$.
Each work-item processes one particle: obtain the global ID, return early if it is outside numParticles, read the current velocity, compute the updated velocity using the constant acceleration g and time step dt, and write the result to the output array. A bounds check is required because the global NDRange may be padded when experimenting with different work-group sizes.

c) In the host code (`02_gravity.cpp`) set the gravity vector g and the time step dt for the simulation (for example, gravity acting in the negative *y* direction with a small-time step). Then create the `apply_gravity` kernel and set all kernel arguments in the correct order, ensuring that `numParticles` is passed to enable bounds checking inside the kernel.

d) Benchmark the kernel that you have created using both the automatic local size (letting OpenCL choose) and your manually selected local sizes. Record the runtimes, state which configuration performs best, and briefly explain why (e.g., better alignment with the device's preferred work-group multiple, improved occupancy, or reduced scheduling overhead).

2.2. Acceleration from Force

According to Newton's Second Law, acceleration is proportional to force and inversely proportional to mass.

$$a = \frac{F}{m}$$

In this task we introduce heterogeneous particle properties by assigning a different (but never zero) mass to each particle, while remaining a Map operation. All required data for this task is created and initialised in the host program, and the host-side code (`03_acceleration.cpp`) is fully implemented. In the host code, a force vector and a mass value are generated for every particle. The masses are kept within a small range (approximately 1.0 to 2.0) to ensure Newton's law remains valid, the simulation stays numerically stable, and the effect of varying mass on acceleration can be clearly observed without destabilising the system. The following snippet shows how the force and mass arrays are constructed in the host program before being transferred to the device:

```
std::vector<cl_float3> forces(numParticles);
std::vector<float> masses(numParticles);

for (int i = 0; i < numParticles; ++i) {
    float vx = velocities[3*i + 0];
    float vy = velocities[3*i + 1];
    float vz = velocities[3*i + 2];

    // Force proportional to velocity
    forces[i].x = 0.1f * vx;
    forces[i].y = 0.1f * vy;
    forces[i].z = 0.1f * vz;

    // Varying, non-zero mass
    masses[i] = 1.0f + 0.01f * (i % 100);
}
```

These arrays are then copied to device memory and passed to the OpenCL kernel, which computes acceleration using ($a = F/m$) for each particle. Since all setup, data preparation, and kernel execution logic are already implemented in the host program, no further host-side changes are required for this task. However, your tasks are to:

a) First, compile and execute the program (`03_acceleration.cpp`). You will notice that it terminates with the following error, which occurs because the `compute_accel` kernel has not yet been implemented in the device (kernel) code (`acceleration.cl`):

```
terminate called after throwing an instance of 'cl::Error'
 what():  clCreateKernel
```

b) Now, write the device (kernel) code for computing acceleration. Implement an OpenCL kernel called `compute_accel` that performs a **parallel Map**: each work-item processes one particle and computes its acceleration using $a = F/m$, writing the result to the output acceleration array. This can be implemented in `acceleration.cl`, which you will find in the `kernels/` folder.

2.3 Speed Magnitude

A template for both the kernel code (`speed.cl`) and the host code (`04_speed.cpp`) is included in the downloadable workshop ZIP on Blackboard. The template does not run when compiled but has enough comments showing exactly what you must implement. Complete the following task:

a) Open `kernels/speed.cl` and implement the `compute_speed` kernel to compute the speed (magnitude) for each particle:

$$\| v \| = \sqrt{v_x^2 + v_y^2 + v_z^2}$$

Implement this as a **map of a sequence**: one work-item per particle (map), with the "**square–sum–square root**" operations performed sequentially inside the work-item. Include a bounds check `if (id >= numParticles) return;` to support padded global sizes.

b) Open `04_speed.cpp` and ensure the host-side setup matches the kernel argument order. This file should look very similar to previous examples (context, queue, buffers, program build, kernel args, NDRange launch), so you can reuse the same structure and only adjust filenames, buffer types, and kernel name as needed.

c) Build and run the program, then verify correctness by checking the printed sample speeds for the first few particles.

d) Run the profiling loop with AUTO and the manual local sizes provided in the template (`try 0 (AUTO), 32, 64, 128`), record the runtimes, and briefly comment on which configuration performs best and why.

2.4. Update Particle Lifetimes

A template for both the kernel code (`lifetime.cl`) and the host code (`05_lifetime_updates.cpp`) is included in the downloadable workshop folder. The template does not run as-is but contains enough comments to guide your implementation. Complete the following task:

a) Open `kernels/lifetime.cl` and implement the `update_lifetimes` kernel to reduce each particle's lifetime by dt. The kernel should also mark each particle as alive (`1`) or dead (`0`) depending on whether its lifetime remains positive:

- This introduces conditional logic in the Map pattern — a common feature in real-world particle simulations (e.g., smoke, sparks, debris).
- Implement this as a map of a sequence: one work-item per particle. Include a bounds check: `if (id >= numParticles) return;` to support padded global sizes.

b) Open `UpdateLifetimes.cpp` and ensure the host-side setup matches the kernel argument order. This file should follow the same structure as previous examples (context, queue, buffers, program build, kernel args, NDRange launch), so you can reuse the same patterns and only adjust filenames, buffer types, and kernel name as needed.

c) Build and run the program, then verify correctness by checking the number of alive particles after a time step.

d) Run the profiling loop with AUTO and the manual local sizes provided in the template (`try 0 (AUTO), 32, 64, 128`).

Performance Analysis

Profile one or more of the above physics kernels on the **large dataset** using at least three different work-group sizes. Compare results with the OpenCL default configuration. Which size produced the best performance? How do CPU and GPU results differ?

3 Parallel Pattern – Stencil (Week 7 - 8)

A stencil pattern takes multiple data inputs from a pre-defined neighbourhood and combines them into a single value (see Fig. 5). Like the map pattern, each task/work item is independent.

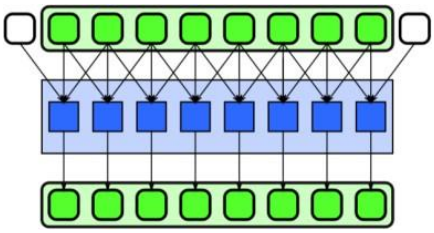

Figure 5 The Stencil parallel pattern (after http://parallelbook.com/sites/parallelbook.com/files/SC13_20131117_Intel_McCool_Robison_Reinders_Hebenstreit.pdf).

3.1 1D Stencil

This task uses the host code (`06_average_filter.cpp`) and the kernel code (`avg_filter.cl`). The kernel file contains a function called `avg_filter_valid`, which applies an averaging filter over a local window of size `k`. For each position, it computes the average of the value at the centre together with its `k/2` neighbouring values on the left and `k/2` neighbouring values on the right. This is a basic smoothing operation, commonly used to reduce noise in signals (for example, an audio waveform). You have been provided with template files on Blackboard, and your task is to complete the following.

a) Adapt the host code so that it can run the kernel and test its operation using different input values. Two datasets are provided in the dataset folder (`dataset/small_signal.bin` and `dataset/large_signal.bin`). The filter parameter (kernel size) is k = 5, which must be positive and odd.

b) Modify the code so that it uses a window of size k = 11. Describe how this larger window affects the smoothness of the signal.

c) Currently, the kernel in `avg_filter.cl` has a valid-only boundary conditions for the 1D averaging filter (`if (gid < r || gid >= N - r) return;`). It does not compute output values at the boundaries. Could you modify the codes to: 1) Clamp - so that when the filter window goes past the signal boundary, you **reuse the nearest valid sample**, that is out-of-range indices are **clamped** into `[0, N-1]`, and 2) Zero-pad - when the filter window goes past the boundary, **missing samples are treated as zero**.

*HINTS: **Clamp:** when the filter window goes out of bounds, you force the index back into the valid range so every output position is still computed. The two key ideas are to protect against extra work-items with `if (gid >= N) return;` and, inside the loop, to correct each out-of-range access using `idx = clamp(idx, 0, N - 1);`. This means the loop always runs k times, the divisor stays k, and edge samples are reused instead of skipped. **Zero-pad:** when the window goes out of bounds, missing samples are treated as zero, but the output is still computed. You again guard against extra work-items with `if (gid >= N) return;`. Inside the loop, you only add a value to the sum when the index is valid, typically using a bounds check like `if (idx >= 0 && idx < N)`, and do nothing otherwise. The sum is still divided by k, but out-of-range positions contribute nothing.*

3.2 2D Stencil

So far, we have only worked with one-dimensional arrays. In certain applications, however, it is more convenient to specify the problem in more than one dimension. For example, when dealing with digital images it is easier to refer to pixel x and y coordinates, rather than their location in a 1D array. Kernel dimensionality is specified within a kernel execution command. In the 2D case, the first parameter of NDRange specifies the number of columns (width), and the second one the number of rows (height). For example, to call a 2D kernel with data arranged into a 5x2 array, the following modification to the host code is required:

```
queue.enqueueNDRangeKernel(kernel_add, cl::NullRange, cl::NDRange(5, 2),
cl::NullRange);
```

The global indices are now arranged into two dimensions and the `get_global_size(d)` function will return the number of elements along the dth dimension. In our example, for `d=0` this value is 5 and for `d=1` it is 2.

Some additional information

For this purpose, we will use some simple applications in digital image processing since the result of operations on digital images are easily visible and provide instant feedback. This is, however, not, by any means, a tutorial on digital image processing, and we will only use these examples as illustrations to generic problems in parallel processing.

The remainder of the tasks are set up to enable I/O operations involving digital images. We will use a framework based on one lightweight multi-platform image processing library called CImg to enable simple visualization and interaction with the user. A digital image is represented as a byte array (CImg<unsigned char>) containing pixels storing the intensity level for each colour channel. CImg stores pixels for each colour channel (RGB) separately, so that the pixels for red channel are stored first, followed by the green and then blue channel (see Fig. 3). The image has width and height properties defining rows and columns, and the spectrum property which specifies the number of colour channels (3 in our case). We will load a PNG image using CImg. To support PNG files, CImg must be compiled with PNG support enabled, which requires linking against libpng and zlib. If these libraries are not available at compile time, CImg will still compile, but loading PNG images will fail at runtime or produce an error. On Windows, compile with the following linker flags: -lpng -lz.

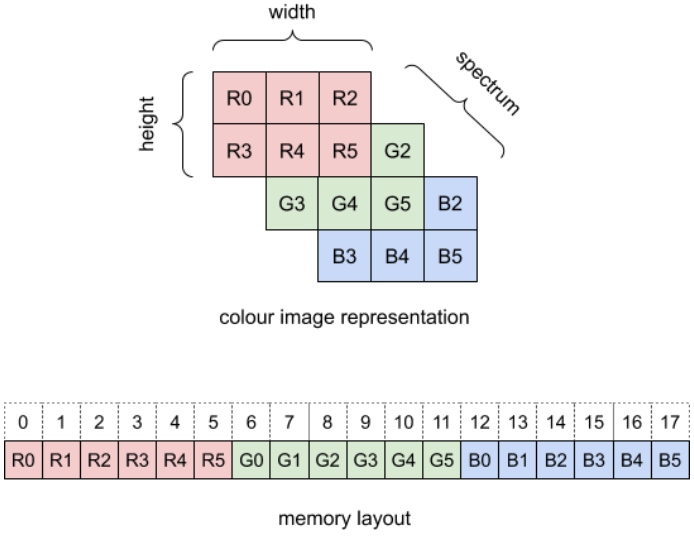

Figure 3 An example CImg image.

In the remaining tasks, the input and output are images rather than simple vectors. Before running the code for the next task, it is important to study the host program carefully and note how it differs from the earlier vector-based examples. Two images are provided with the code: a smaller image, `dataset/opencv_small.png`, which is intended for development and debugging, and a larger image, `dataset/opencv_large.png`, which is used for evaluation and performance analysis. In most cases, the kernel code treats the image as a single long one-dimensional array, as illustrated in Fig. 3, and processes the image byte by byte without explicitly distinguishing between the different colour channels.

3.2.1 Averaging filter

Let us now look at an averaging filter in 2D. The kernel code `kernels/avg_filter.cl` has a function `avg_filterND` a similar function to the 1D averaging filter developed in the previous task. The filter calculates an average of a `kxk` local window. This operation results in a blurred (smoothed) version of the input image.

1. Adjust the **host code** such that the 2D averaging filter kernel can be run on the input image (`dataset/opencv_small.png`). Ensure that `image.width()`, `image.height()` and `image.spectrum()` are specified as values for kernel dimensions. Inspect the kernel code (note how 2D indices are used in the loop), run the code and visually test the results.

2. Modify the host code such that it uses a window of size 11x11 or higher. How does this larger window affect the results?

Note the Kernel dimensionality that is specified within a kernel execution command

```
cl::NDRange global(static_cast<size_t>(width), static_cast<size_t>(height),
static_cast<size_t>(channels));
```

Also note that `CImg` is used to manage image I/O and data layout. The image is first loaded directly from a file using the `CImg` constructor, which automatically reads the PNG image and provides access to pixel values via (x, y, c) indexing. Since the OpenCL kernel expects planar RGB data, the `CImg` image, which is accessed in (x, y, c) form, is explicitly converted into a planar buffer on the host using the indexing scheme `x + y * width + c * (width * height)`. After the kernel execution, the processed planar output buffer is converted back into a `CImg` object using the same indexing relationship in reverse, allowing the result to be displayed easily using `CImgDisplay`. This approach keeps image loading, visualization, and data conversion clearly separated while ensuring compatibility between `CImg's` image representation and the planar memory layout required by the OpenCL kernel.

3.2.2 Convolution filter

The averaging filter can also be implemented as a generic image convolution operation. Image convolution uses a rectangular mask of weighting values (also called a convolution kernel) applied to every pixel and its neighbourhood. The result of this operation is a weighted sum of pixel values in the local neighbourhood. Each pixel is multiplied by the corresponding mask value and added together (see https://en.wikipedia.org/wiki/Kernel_(image_processing) for an illustrative example). Our averaging filter can be simply expressed as a `kxk` mask with all entries equal to `1/(k*k)`. The result of such a convolution operation is the same as for the `avg_filterND`. There are many other combinations of mask values which result in different image processing operations including sharpening, soft blurring, edge detection, etc. There are several examples of such masks listed in the following Wikipedia entry: see https://en.wikipedia.org/wiki/Kernel_(image_processing). For this

tasks you have been provided with a simple implementation of a `kxk` convolution filter in the kernel file (`kernels/convolution_filter.cl`). You will notice that the mask is provided as an additional input argument and initialised in the host code (`08_convolutionND.cpp`). To run the convolution example, you need to modify the host code by uncommenting the respective code lines responsible for allocating and transferring the mask values to the device.

1. Study the averaging convolution kernel and run the code. Compare the blurring results with those obtained from the `avg_filterND` kernel. You may want to use screenshots to perform direct visual comparison.

2. Try out other convolution masks such as those specified in the Wikipedia article.

3.2.3 Additional Tasks

The following tasks are designed for eager students who have finished all the above tasks and would like an extra challenge - the instructions provided are deliberately sparse! These tasks are optional and not necessary to progress further in the following tutorials.

1. Write a kernel function `brightness` that, in addition to input and output images, also accepts a separate integer parameter called `beta`. This value should be added to the intensity value of each colour component. This is a simple brightness adjustment function. Test your program on a larger image for different values of the brightness parameter. You have been provided with starter templates for the host codes (`09_brightness.cpp`) and kernel codes (`brightness.cl`), but they are extremely sparse.

2. Write a kernel function called `contrast` that adjusts the contrast of an input image. The contrast value should be specified as a `float` parameter, which should then be multiplied by each colour component. Try different values of the parameters on your input image. You have been provided with starter templates for the host codes (`10_contrast.cpp`) and kernel codes (`contrast.cl`), but they are extremely sparse.